Topic 6: Queues and Deques

6.1 Queue ADT

- A Queue is a First-In First-Out (FIFO) data structure
- Operations: enqueue (add to rear), dequeue (remove from front), front (view front), isEmpty
- Analogies: ticket queue, print spooler, BFS traversal
- All operations are O(1)

6.2 Array-Based Circular Queue

- Use a fixed array with front and rear indices and a count
- Wrap around using modulo: rear = (rear + 1) % capacity
- Avoids the 'shifting' problem of a naive linear array queue

```
class CircularQueue {
    int data[10], front=0, rear=-1, count=0;
    const int CAP = 10;
public:
    void enqueue(int v) {
        if (count == CAP) { cout << "Full"; return; }
        rear = (rear + 1) % CAP;
        data[rear] = v; count++;
    }
    int dequeue() {
        if (count == 0) return -1;
        int v = data[front];
        front = (front + 1) % CAP; count--;
        return v;
    }
};
```

6.3 Linked-List-Based Queue

- Maintain head (front) and tail (rear) pointers
- Enqueue: add node at tail -- O(1)
- Dequeue: remove node from head -- O(1)
- No capacity limit

6.4 Priority Queue

- Elements are dequeued in priority order, not arrival order

- Implementation: binary heap (standard) or sorted array
- std::priority_queue<int>: max-heap by default -- top() is always the largest
- Min-heap: priority_queue<int, vector<int>, greater<int>>
- Used in Dijkstra's shortest path, task scheduling, Huffman coding

```
#include <queue>
priority_queue<int> maxH;
maxH.push(3); maxH.push(1); maxH.push(9);
cout << maxH.top();  // 9
```

6.5 Deque (Double-Ended Queue)

- Supports insert and delete at both front and rear in O(1)
- std::deque<T> in the STL
- push_front, push_back, pop_front, pop_back
- Used to implement sliding window maximum, palindrome checking

---